

BruceButBetter-Modul

© Louis Havet-Quillivic & Joachim Willner · Version 0.4 · 21. September 2026

1. Einleitung

Dieses Dokument beschreibt den Start des Projekts «BruceButBetter-Modul»: Louis und Joachim wollen ein eigenes Hardware-Modul entwickeln, das sich an BruceButBetter orientiert – einem Downstream-Fork der Bruce-Firmware für ein selbstgebautes ESP32-S3-Multitool mit Flipper-Zero-ähnlichen Funktionen. Es dient als gemeinsamer Ausgangspunkt und wird mit fortschreitendem Projekt ergänzt. Das Projekt wird im GitHub-Repository github.com/jwillner/Bruce-But-Better-by-JoWi-LoHa geführt.

1.1 Ausgangslage

BruceButBetter ist ein bestehendes Open-Source-Projekt (github.com/Yoursel71/BruceButBetter), das auf einem ESP32-S3 N16R8 mit 1,3"-OLED-Display aufbaut und optional Module wie CC1101 (Sub-GHz), PN532 (NFC/RFID), zwei NRF24-Funkmodule, einen Si5351-Signalgenerator, IR-Sender/-Empfänger, microSD und einen Buzzer unterstützt. Vorbild ist der Flipper Zero. Statt das dortige Referenzdesign nachzubauen, wollen wir eine eigene Platine entwerfen und dabei selbst festlegen, welche Module verbaut werden.

1.1.1 Ziel des Projekts

Ziel der ersten Phase ist eine eigene, funktionsfähige Leiterplatte mit ESP32-S3, Display und einer noch festzulegenden Auswahl an Modulen, auf der die Bruce-/BruceButBetter-Firmware lauffähig ist. Der Eigenbau soll Erfahrung im Platinendesign (KiCad) vermitteln und Raum für spätere eigene Anpassungen an Hardware wie Firmware schaffen.

1.2 Leitgedanke

Wir bauen nicht das Referenzdesign nach, sondern entwickeln ein eigenes, zur Firmware kompatibles Layout. Umfang und Modulauswahl richten sich nach dem, was wir tatsächlich nutzen wollen – lieber ein kleineres, sauber funktionierendes Modul als ein Nachbau mit ungenutzten Funktionen.

2. Funktionsmuster-PCB

Bevor wir das Gesamtlayout finalisieren, bauen wir als Erstes ein Funktionsmuster-PCB, auf dem sich jede Baugruppe einzeln bestücken und testen lässt. Die folgende Übersicht zeigt die dafür vorgesehenen Baugruppen mit einer kurzen Funktionsbeschreibung und einem Link auf die jeweilige Herstellerbeschreibung. Die Bilder sind vereinfachte, einheitlich gestaltete Symbole – das genaue Bauteilfoto findet sich auf der verlinkten Herstellerseite.

BruceButBetter-Modul

© Louis Havet-Quillivic & Joachim Willner · Version 0.4 · 21. September 2026

	Baugruppe	Funktion	Herstellerbeschreibung
	ESP32-S3 (Espressif)	Hauptprozessor mit WLAN/BLE, steuert Display und alle Module	espressif.com/.../esp32-s3
	OLED-Display 1,3" (Controller SSD1306)	Anzeige für Menü und Status	solomon-systech.com/.../ssd1306
	CC1101 (Texas Instruments)	Sub-GHz-Funk (315/433/868/915 MHz) für Fernbedienungen und Sensoren	ti.com/product/CC1101
	PN532 (NXP)	NFC/RFID bei 13,56 MHz lesen und schreiben	nxp.com – PN532-Datenblatt
	NRF24L01+ (Nordic Semiconductor), 2x	2,4-GHz-Funk für Sniffing/Angriffe im ISM-Band	nordicsemi.com/.../nRF24L01
	Si5351A (Skyworks)	Programmierbarer Takt-/Signalgenerator	skyworksinc.com/.../Si5351A-B-GT
	IR-Sender/-Empfänger (Vishay TSAL6200 / TSOP38238)	Infrarot-Fernbedienungen senden und empfangen	vishay.com/.../81010 (Sender) vishay.com/.../82491 (Empfänger)
	microSD-Kartensteckplatz (Molex)	Lokaler Speicher für Dateien, Firmware, Payloads	molex.com – microSD-Steckverbinder
	Buzzer (CUI Devices / Same Sky, CMT-8540S)	Akustisches Feedback	cui.com/.../cmt-8540s-smt

3. Planung und Umsetzung

Die folgende Übersicht zeigt die geplanten Arbeitsschritte. Termine sind zum jetzigen Zeitpunkt noch offen und werden ergänzt, sobald der Modulumfang feststeht.

BruceButBetter-Modul

© Louis Havet-Quillivic & Joachim Willner · Version 0.4 · 21. September 2026

Schritt	Zuständig	Termin
Entwicklungsumgebung auf Proxmox einrichten (Linux-VMs, Tools unter Versionskontrolle)	Louis & Joachim	offen
Referenzprojekt & Firmware sichten (Bruce/BruceButBetter, Schaltplan, BOM)	Louis & Joachim	offen
Modulumfang für Version 1 festlegen	Louis & Joachim	offen
Bauteile für Funktionsmuster-PCB beschaffen (siehe Kapitel 2)	offen	offen
Schaltplan und Platine in KiCad entwerfen	offen	offen
Prototyp fertigen und bestücken lassen	offen	offen
Firmware flashen und Module einzeln testen	offen	offen

3.1 Entwicklungsumgebung

Die Entwicklungsumgebung läuft nicht lokal, sondern auf einem PC mit Proxmox: Darauf sind mehrere Linux-VMs eingerichtet, eine davon dient als eigentliche Entwicklungsumgebung, deren installierte Tools unter Versionskontrolle stehen. Für die Arbeit daran verbinden wir uns per VS Code Remote auf diese Linux-VM und nutzen dort Claude Code. Auf einer weiteren Linux-VM läuft Hermes zur Entwicklungsunterstützung. Der Projektcode liegt im öffentlichen GitHub-Repository github.com/jwillner/Bruce-But-Better-by-JoWi-LoHa.

3.2 Zusammenarbeit

Louis (louis.havetquillivic@gmail.com) und Joachim (joachim.willner@gmail.com) planen und entwickeln das Modul gemeinsam. Die konkrete Aufteilung von Hardware- und Firmware-Aufgaben wird beim ersten gemeinsamen Termin festgelegt und danach in diesem Dokument nachgetragen.

3.2.1 Nächste Schritte

Als Nächstes werden das BruceButBetter-Repository und die zugrunde liegende Bruce-Firmware im Detail gesichtet (Referenzschaltplan, Stückliste, unterstützte Module), um daraus den Modulumfang für die erste eigene Version abzuleiten. Anschliessend wird das KiCad-Projekt im Verzeichnis hardware/ angelegt.

4. Ausblick

Nach der ersten funktionierenden Version ist offen, ob das Modul um weitere Funktionen erweitert, in ein Gehäuse integriert oder in einer zweiten Revision überarbeitet wird. Diese Punkte werden nach Abschluss der ersten Iteration entschieden.

Hinweis: Dieses Dokument ist der erste Entwurf und wird mit fortschreitendem Projekt aktualisiert. Termine und Aufgabenverteilung sind aktuell Platzhalter.